

MODUL 3
PENGEMBANGAN APLIKASI MOBILE

Event, State, dan Implisit Intent pada Jetpack Compose

Dosen Pengampu: Muhammad Aminul Akbar, S.Kom., M.T.



Oleh

Yosefina Agustine K. Ofong
NIM. 245150400111030

PROGRAM STUDI SISTEM INFORMASI
DEPARTEMEN SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

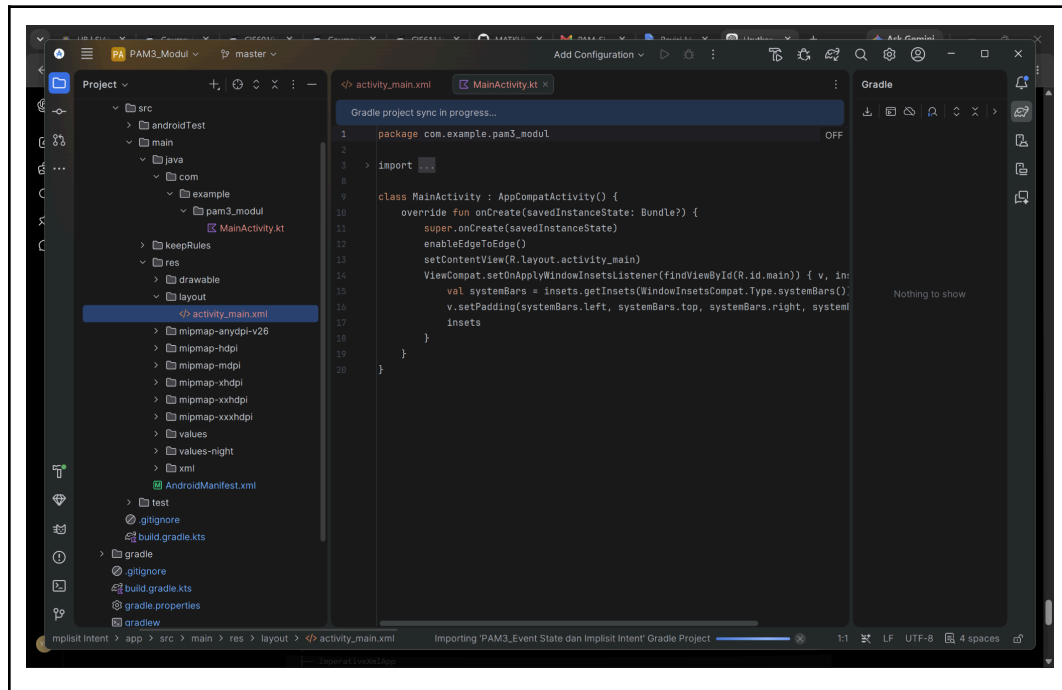
MALANG

2026

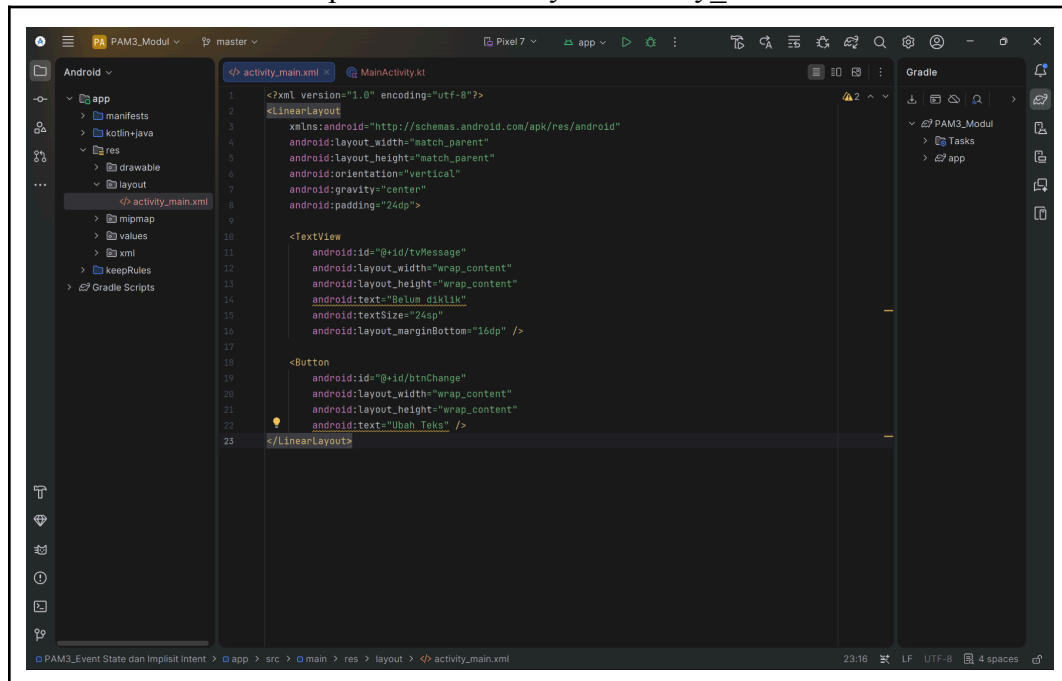
A. Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Project A - XML/View (Imperatif)

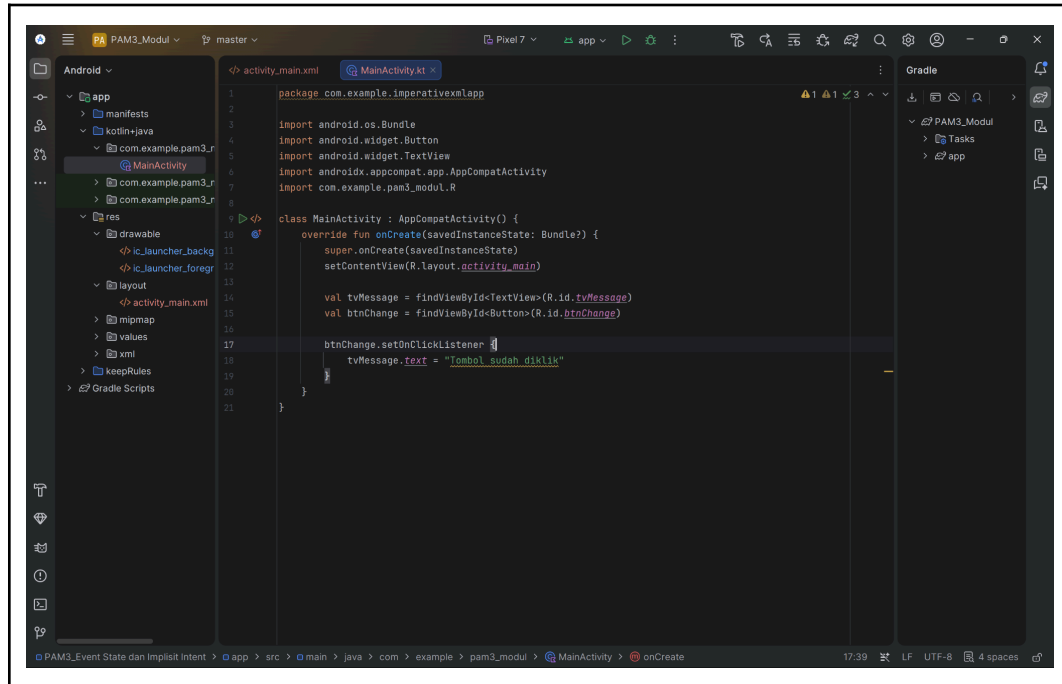
1. Buat project Android Studio baru menggunakan template Empty Views Activity. Beri nama project ImperativeXmlApp dan pilih Kotlin.



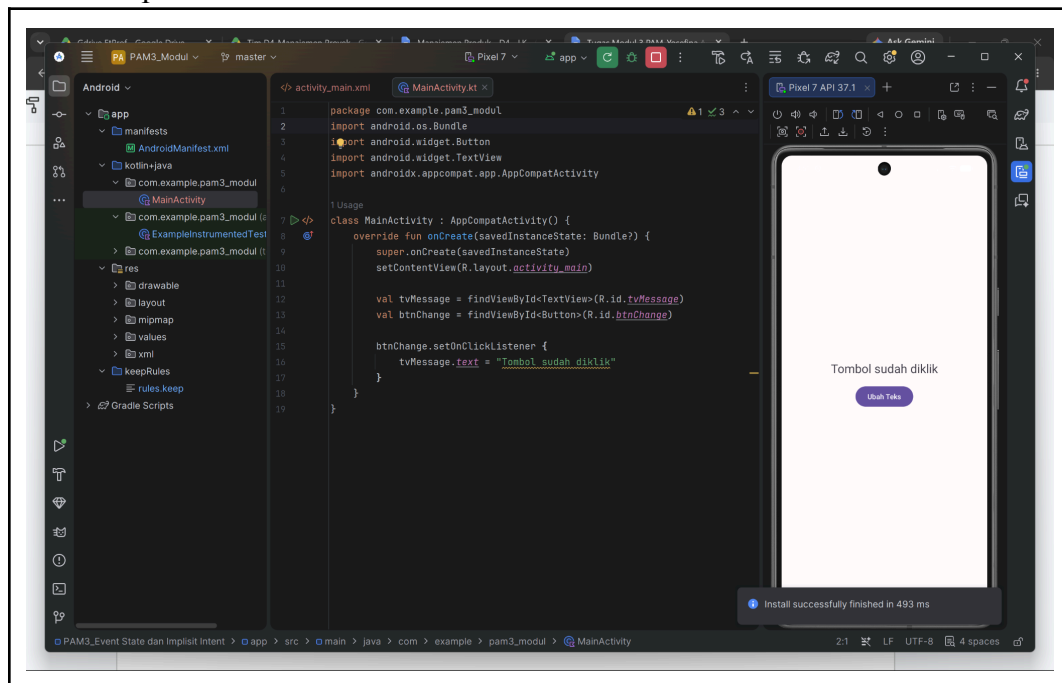
2. Memasukan kode sesuai perintah ke res/layout/activity_main.xml.



3. Memasukan kode sesuai perintah ke MainActivity.kt.

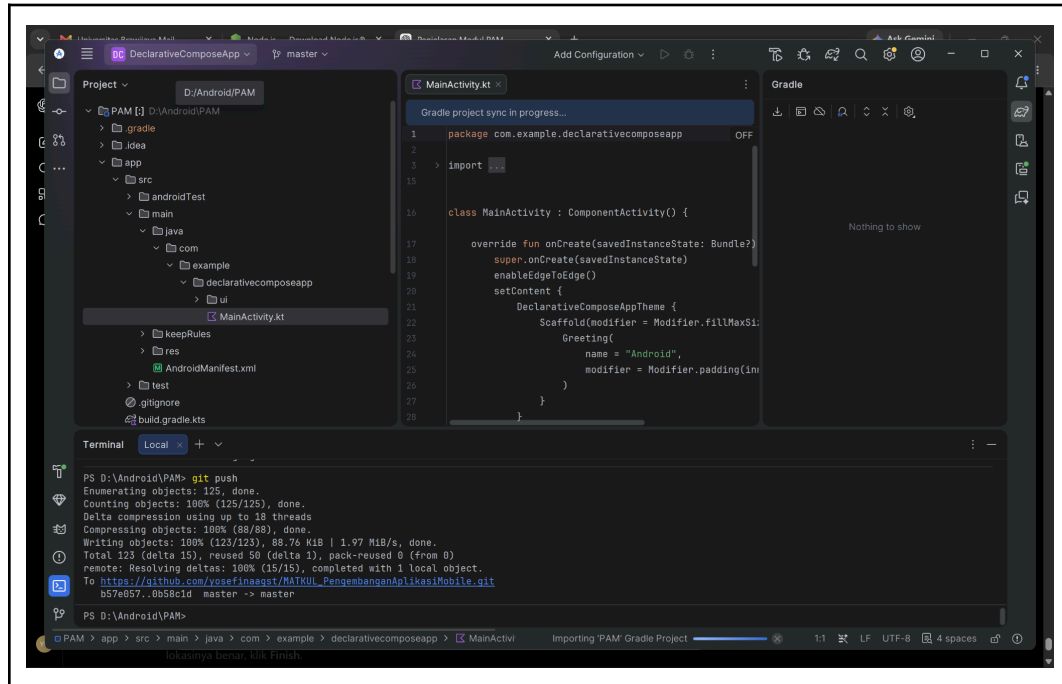


4. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.

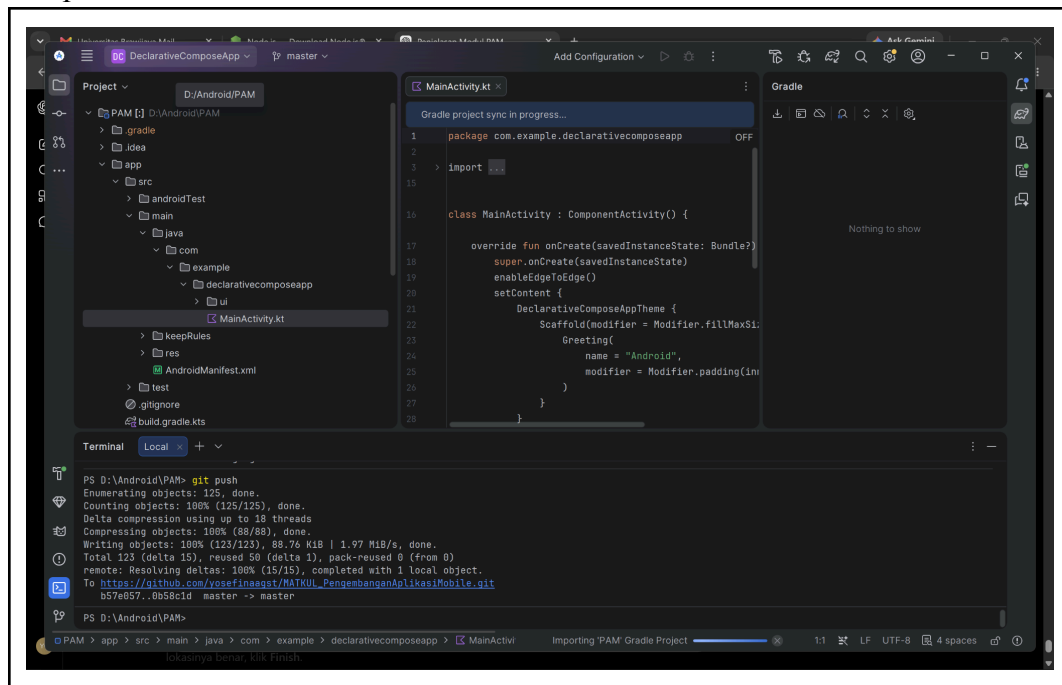


Project B - Jetpack Compose

1. Buat project Android Studio baru menggunakan template Empty Activity untuk Jetpack Compose. Beri nama project DeclarativeComposeApp dan pilih Kotlin.

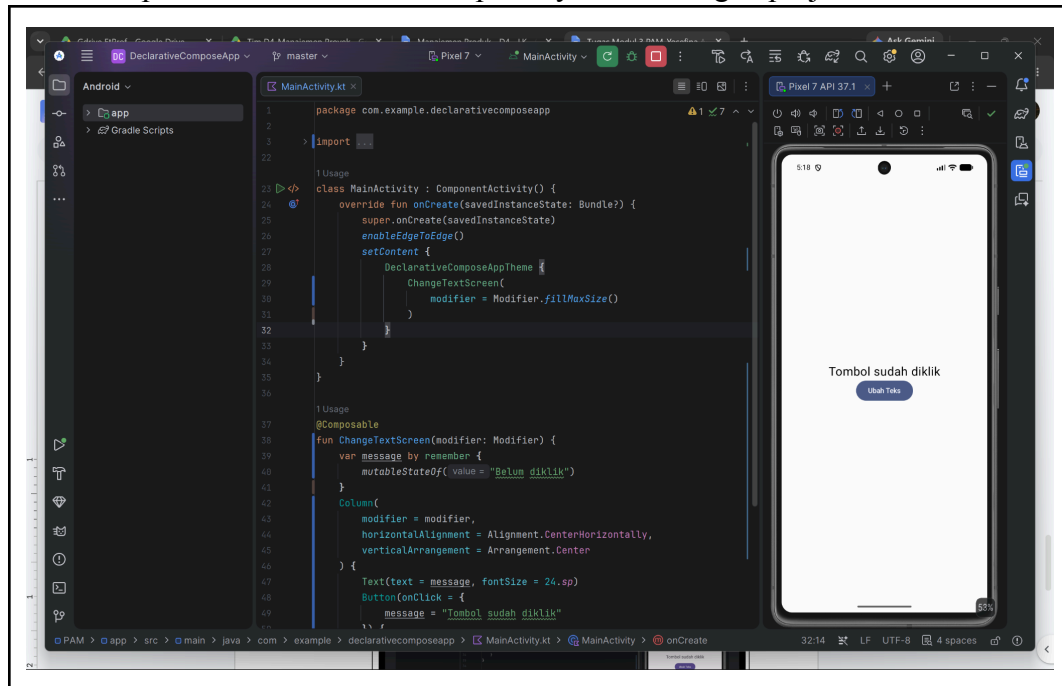


2. Buka MainActivity.kt, pertahankan package project, kemudian gunakan fungsi composable berikut.



3. Tambahkan import yang diperlukan
4. Pada fungsi onCreate(), ubah isi setContent agar memanggil ChangeTextScreen():

5. Jalankan aplikasi. Pastikan hasil tampilannya sama dengan project XML/View.

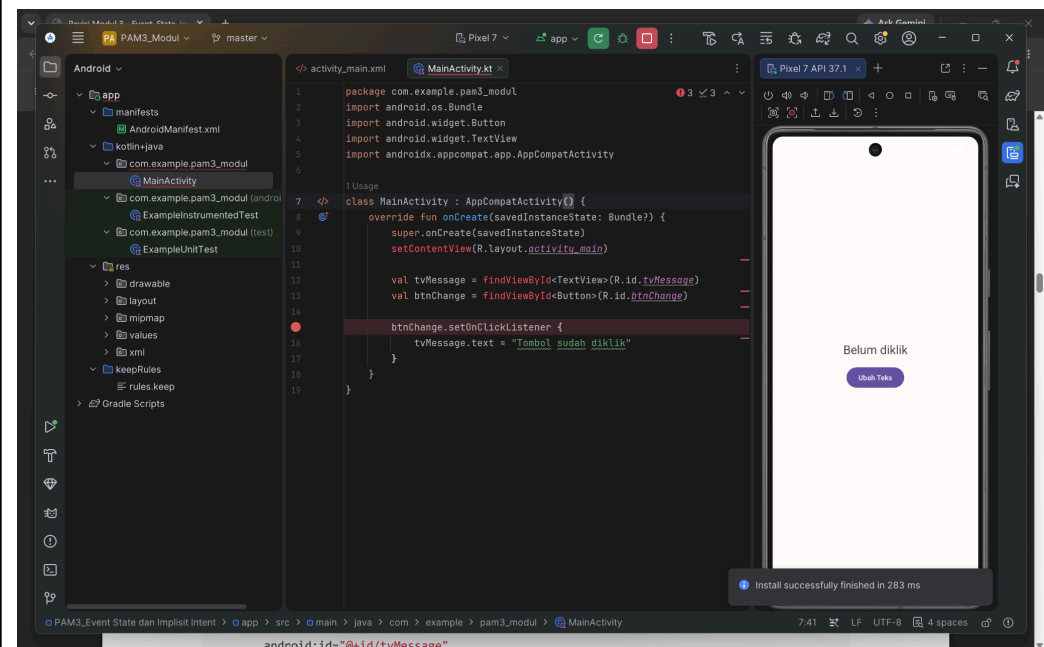


Aktivitas Percobaan

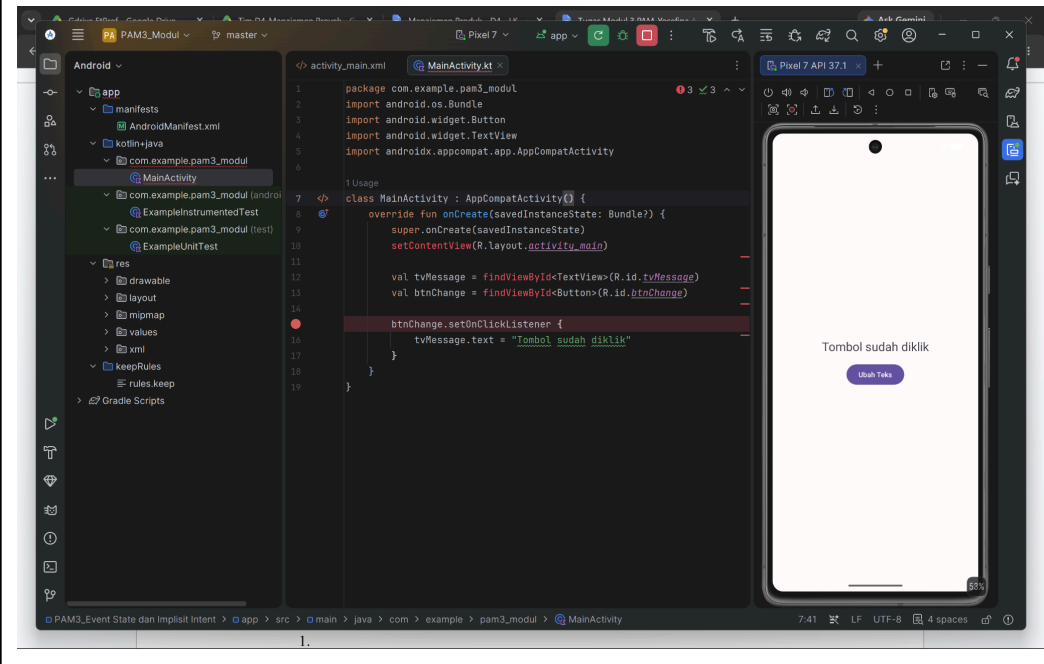
1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Project: Project XML/View [ImperativeXmlApp]

//sebelum di-klik

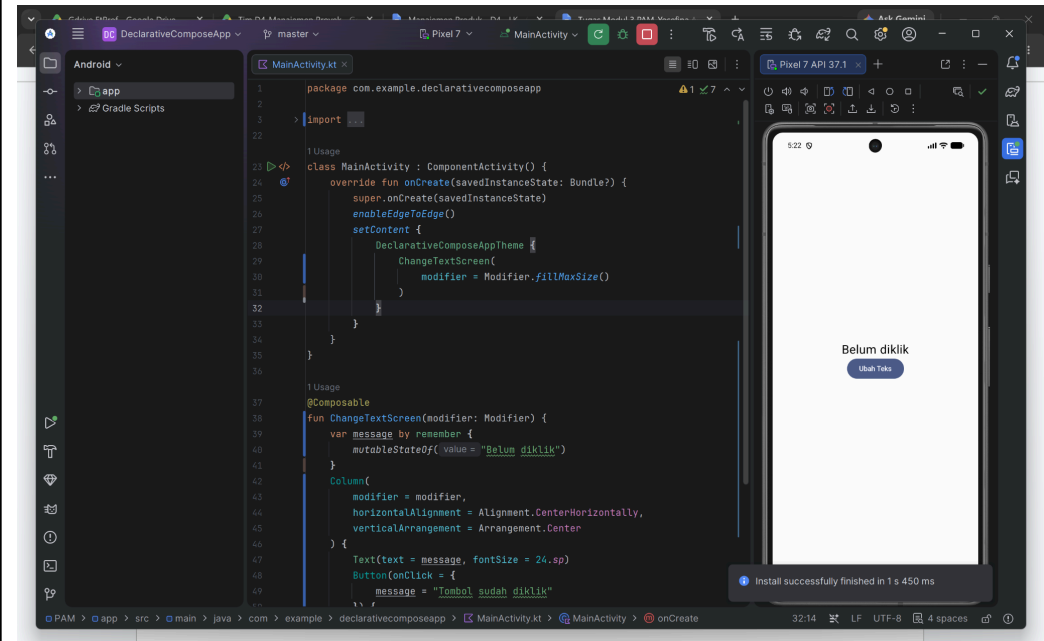


//sesudah di-klik

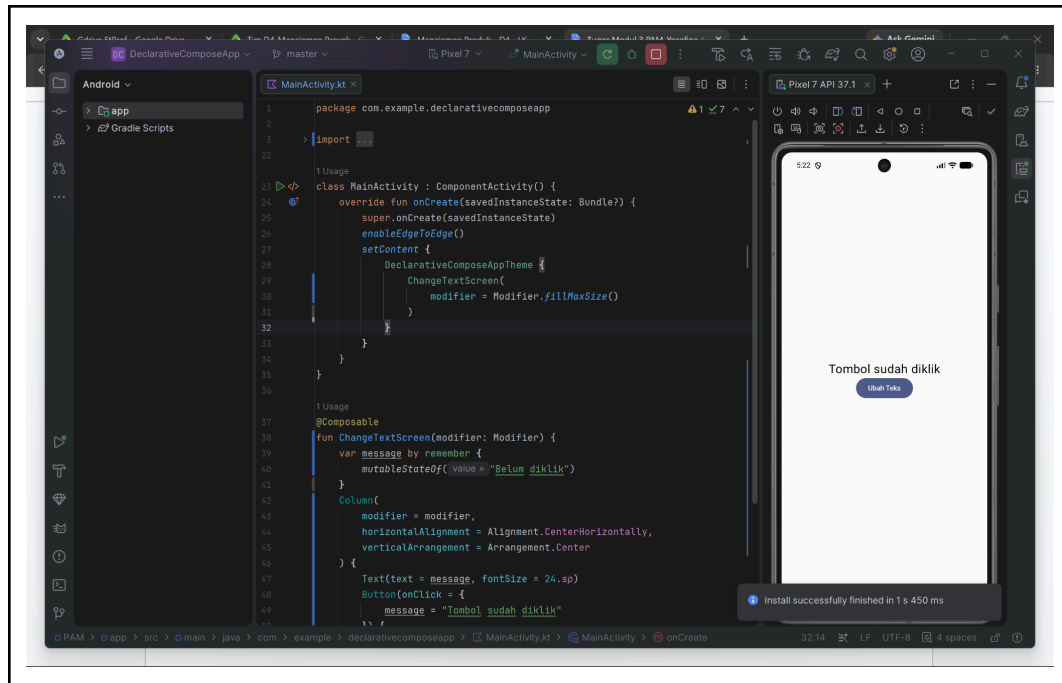


Project Jetpack Compose [DeclarativeComposeApp]

//Sebelum di-klik

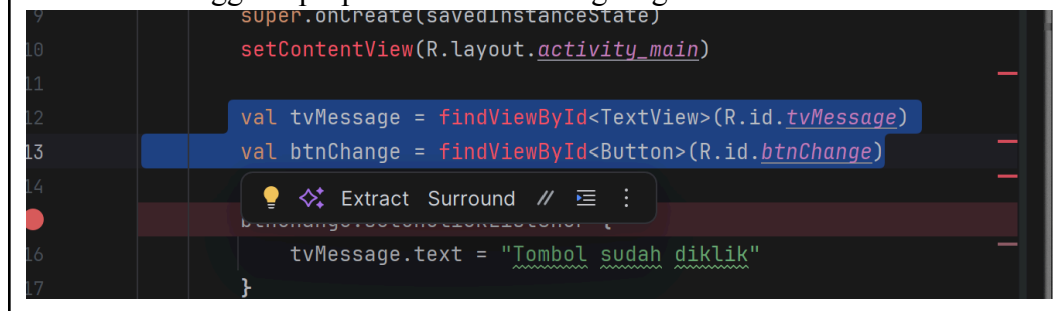


//Sesudah di-klik



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah secara langsung adalah TextView dengan ID tvMessage. Objek tersebut disimpan dalam variabel yang tertera pada highlight gambar berikut. Dan Ketika tombol ditekan, program mengakses objek TextView kemudian mengganti properti text secara langsung.



3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Nilai yang diubah adalah state message.

```

36      1 Usage
37      @Composable
38      fun ChangeTextScreen(modifier: Modifier) {
39          var message by remember {
40              mutableStateOf( value = "Belum diklik")
41          }
42          Column(
43              modifier = modifier,
44              horizontalAlignment = Alignment.CenterHorizontally,
45              verticalArrangement = Arrangement.Center
46          ) {
47              Text(text = message, fontSize = 24.sp)
48              Button(onClick = {
49                  message = "Tombol sudah diklik"
50              }) {
51                  Text("Ubah Teks")
52              }
53          }
54      }
55

```

Ketika tombol ditekan nilai state diubah melalui Message = “Tombol sudah di-klik” lalu komponen Teks mengambil isi yang ditampilkan dari state tersebut (Text(text = message)).

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Pada Jetpack Compose, tampilan dibangun menggunakan pendekatan deklaratif. Programmer tidak mencari dan mengubah objek UI secara langsung. Programmer cukup mendeklarasikan bahwa teks yang ditampilkan berasal dari state message.

Ketika state message berubah, Compose mendeteksi perubahan tersebut dan melakukan recomposition. Oleh karena itu, perintah seperti tvMessage.text atau setText() tidak diperlukan.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Alurnya:

setOnClickListener dijalankan → program mengakses objek tvMessage → properti text pada TextView diubah → tampilan berubah.

Jadi, pada pendekatan XML/View, event klik ditangani menggunakan setOnClickListener. Ketika event terjadi, program mengakses objek TextView melalui variabel tvMessage, kemudian mengubah properti text secara langsung.

Pendekatan ini disebut imperatif karena programmer memberikan perintah secara langsung kepada komponen UI.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Alurnya:

onClik dijalankan → state message berubah → compose mendeteksi perubahan state → terjadi recomposition → Text menampilkan nilai message terbaru.

Jadi, pada Jetpack Compose, event onClick tidak mengubah komponen Text secara langsung. Event tersebut mengubah state message. Perubahan state menyebabkan Compose melakukan recomposition terhadap bagian UI yang menggunakan state tersebut, sehingga teks pada layar diperbarui.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum di-klik"`? Jelaskan penyebabnya.

Tampilan UI tidak akan diperbarui sesuai harapan.

Hal ini terjadi karena:

- Message hanya variabel lokal biasa.
- Compose tidak mengamati perubahan variabel biasa.
- Perubahannya tidak memicu recomposition.
- Jika composable dijalankan kembali, variabel lokal tersebut dapat dibuat ulang dengan nilai awal "Belum diklik".

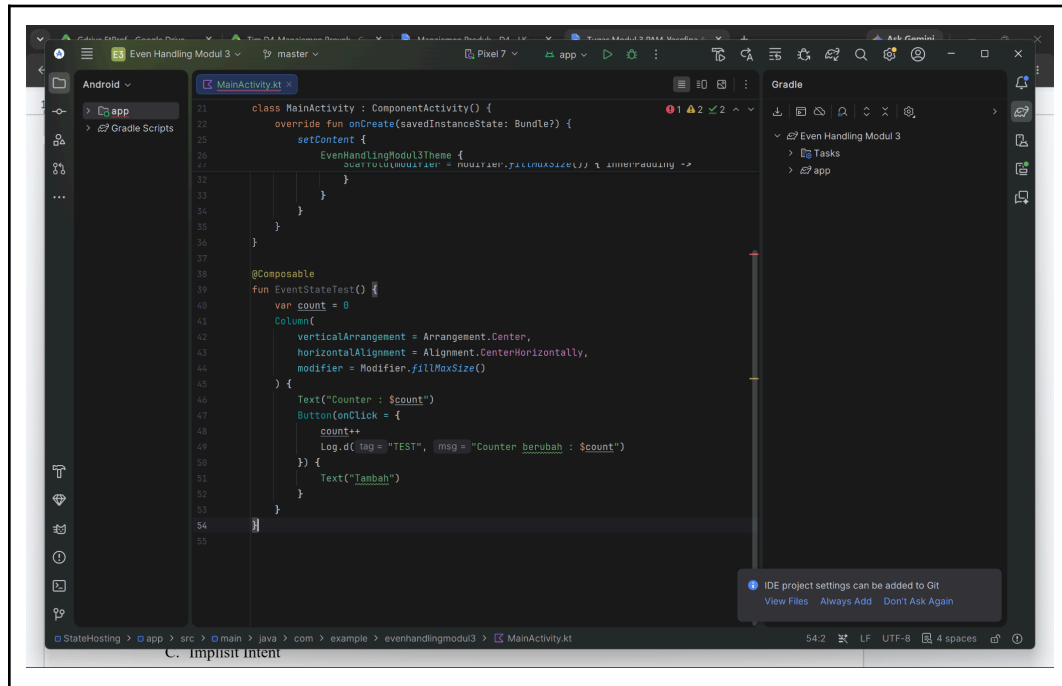
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Pendekatan imperatif memperbarui komponen UI secara langsung melalui referensi objek, misalnya dengan mengubah properti text pada TextView. Sementara itu, pendekatan deklaratif pada Jetpack Compose memperbarui state sebagai sumber data tampilan. Ketika state berubah, Compose akan melakukan recomposition dan memperbarui UI secara otomatis. Dengan demikian, pada pendekatan deklaratif programmer lebih berfokus pada kondisi UI yang ingin ditampilkan daripada langkah-langkah untuk mengubah objek UI secara langsung.

B. Event Handling State

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.

2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity.
3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut.

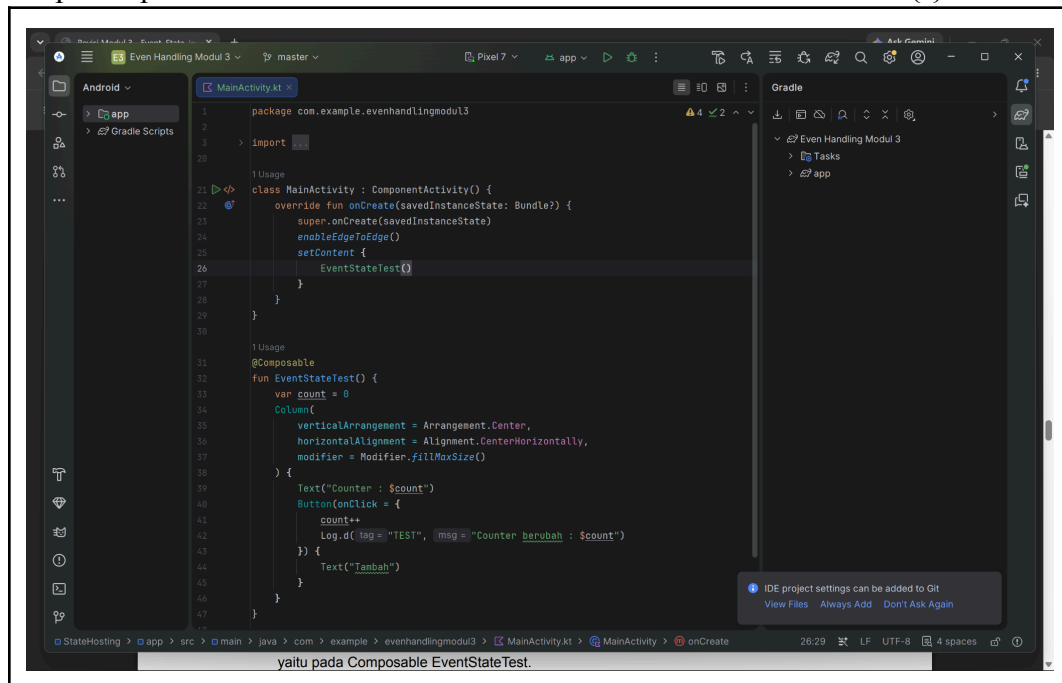


The screenshot shows the MainActivity.kt file in Android Studio. The class MainActivity is defined with an onCreate method. Below the class, a Composable function named EventStateTest is added. The function uses a Column to display a counter and a button to increment it. The code is as follows:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        setContentView(
            EventHandlingModul3Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                }
            }
        )
    }
}

@Composable
fun EventStateTest() {
    var count = 0
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "msg = \"Counter berubah : $count\"")
        }) {
            Text("Tambah")
        }
    }
}
```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Lalu ubah isi setContentView { }



The screenshot shows the MainActivity.kt file in Android Studio. The class MainActivity is defined with an onCreate method. The setContentView method is updated to call EventStateTest. The code is as follows:

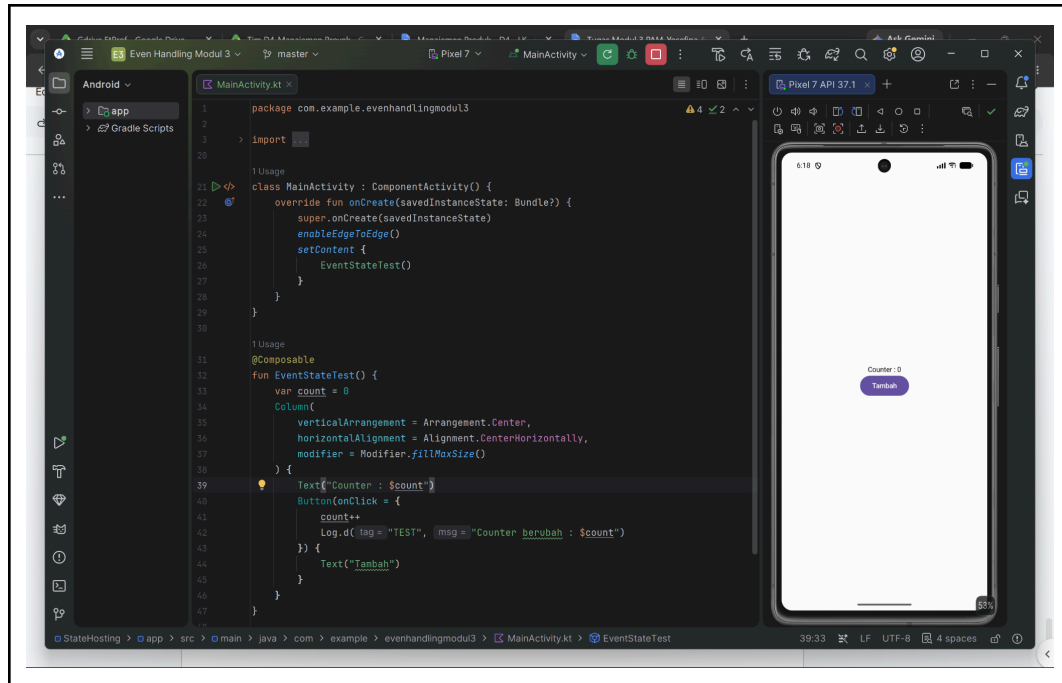
```
package com.example.eventhandlingmodul3

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import androidx.compose.ui.platform.ComposeView

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(
            EventStateTest()
        )
    }
}

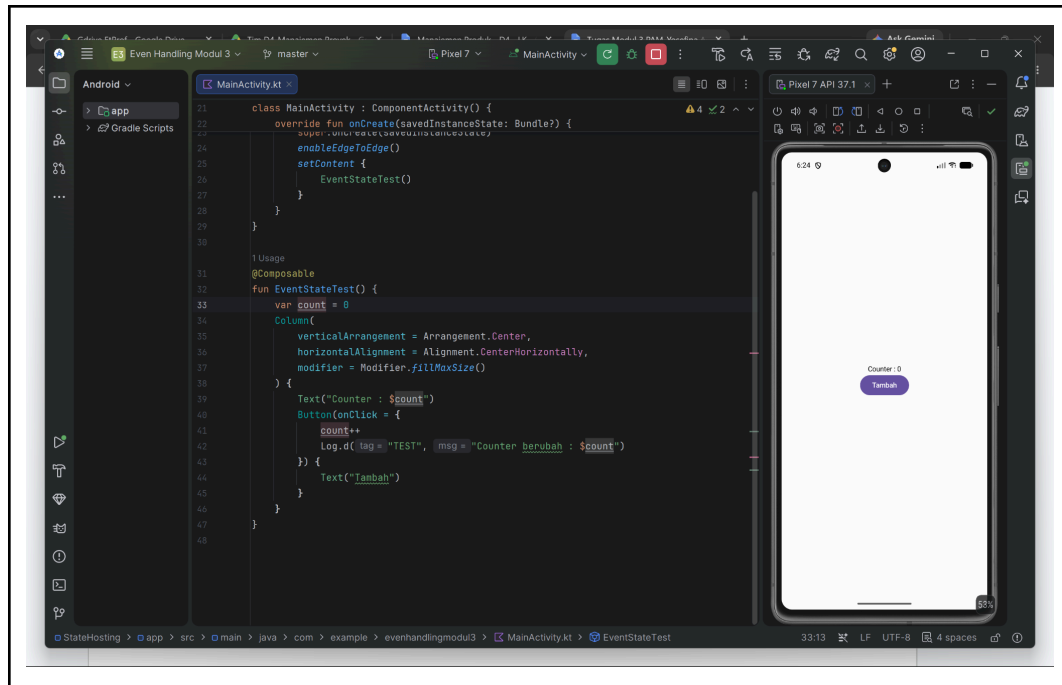
@Composable
fun EventStateTest() {
    var count = 0
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "msg = \"Counter berubah : $count\"")
        }) {
            Text("Tambah")
        }
    }
}
```

5. Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.

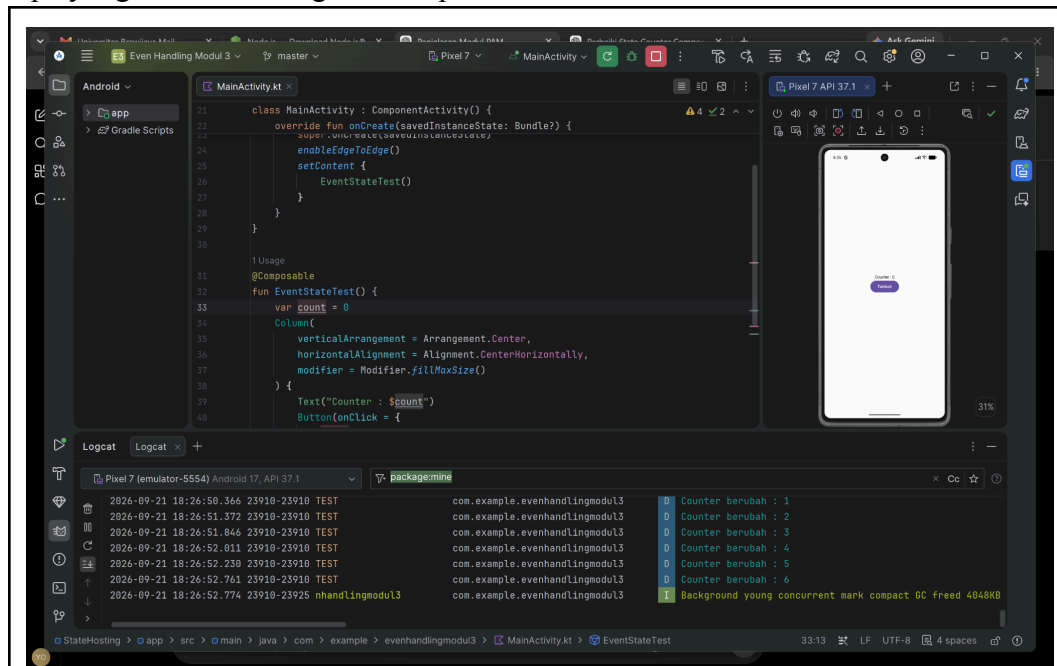


Aktivitas Percobaan.

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?



Saat tombol di-klik, logchat menunjukkan Counter berubah: 1, Counter berubah: 2, dan seterusnya.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

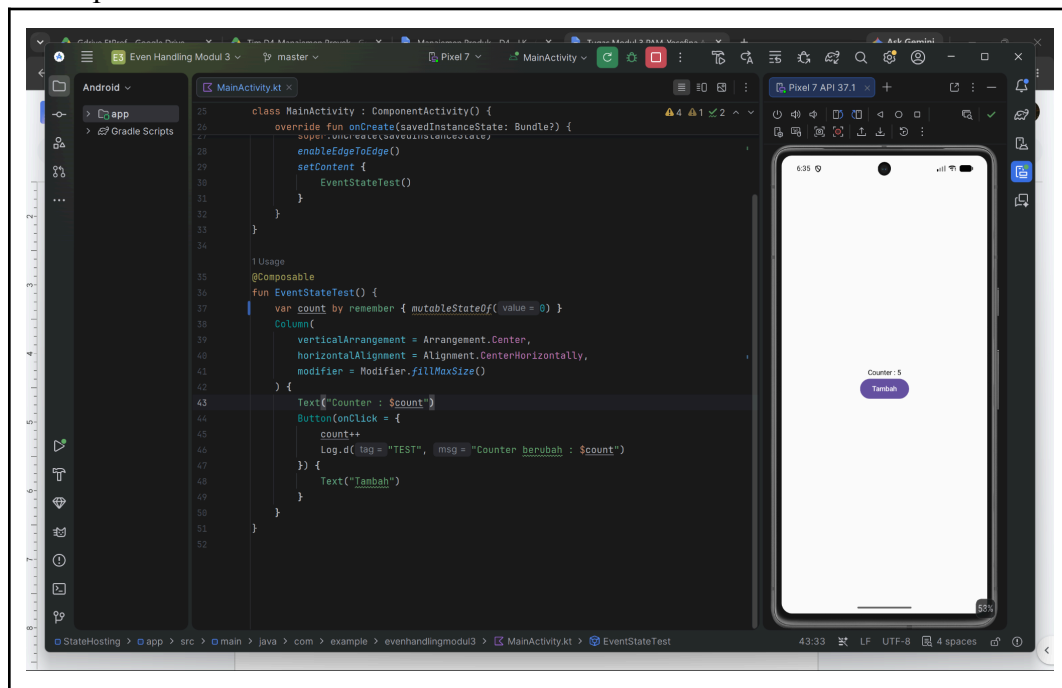
Tidak ada yang terjadi. Tidak ada yang berubah. Tampilan aplikasi tetap menunjukkan "Counter = 0", walau counter terus menunjukkan adanya

tambahan nilai.

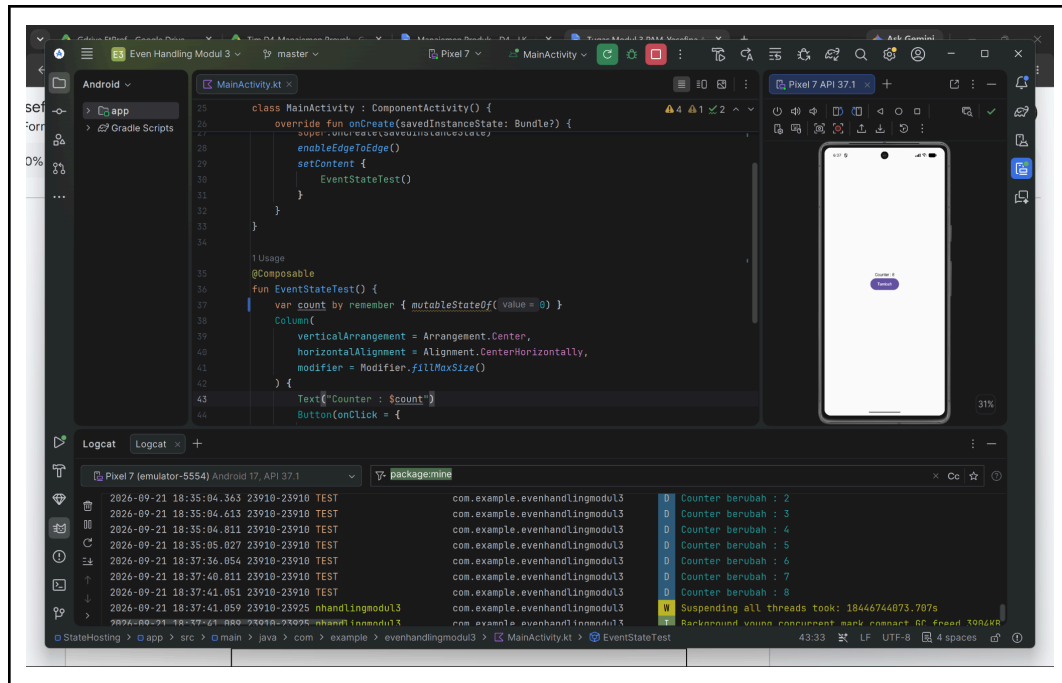
4. Apa yang menyebabkan hal tersebut terjadi?

Hal ini terjadi karena count dibuat sebagai variabel lokal biasa (`var count = 0`), sehingga Compose tidak mengamati perubahan variabel biasa. Oleh karena itu, perubahan count tidak memicu recomposition dan bagian UI berikut tidak dijalankan ulang (`Text("Counter: $count")`)

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable `EventStateTest`.
6. Ubah kode menjadi `var count by remember { mutableStateOf(0) }`
7. Kode lengkap `eventStateTest`
8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan `TEST`. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?



10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Aplikasi menampilkan:
Counter : 1
Counter : 2
Dan seterusnya, bertambah sebanyak jumlah tekanan pada tombol.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

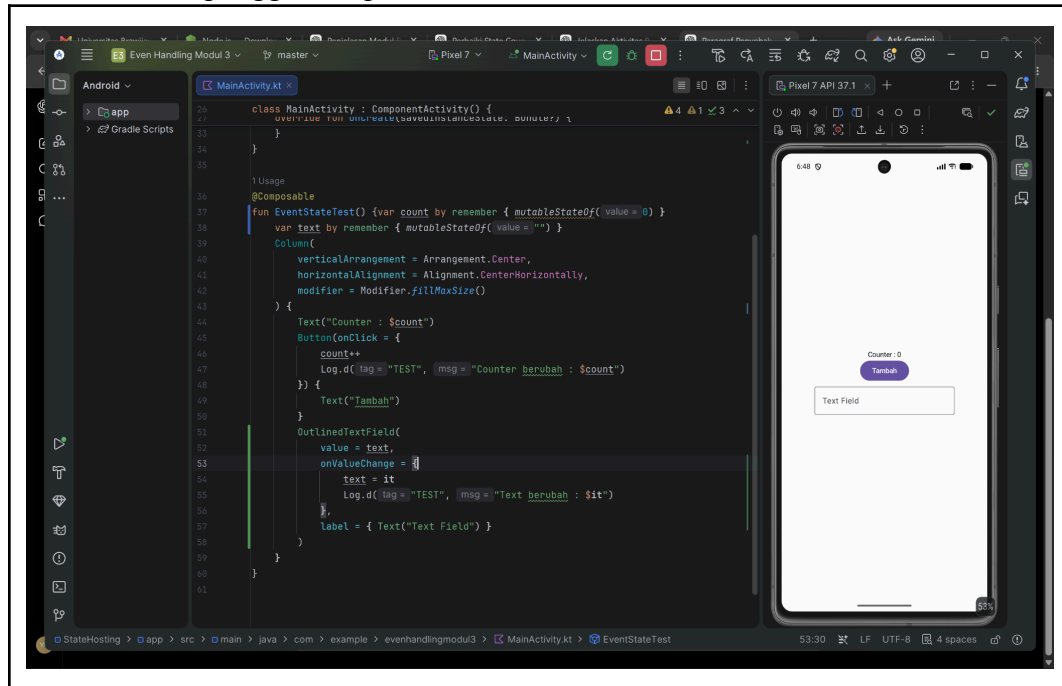
Hal tersebut terjadi karena count masih dideklarasikan sebagai variabel biasa menggunakan `var count = 0`, sehingga variabel tersebut bukan merupakan *observable state* yang dapat dipantau oleh Jetpack Compose. Ketika `count++` dijalankan, nilai count sebenarnya berhasil berubah, sehingga nilai terbaru tetap dapat terlihat pada LogCat melalui `Log.d()`. Namun, Jetpack Compose tidak mendapatkan sinyal bahwa terdapat perubahan data yang mengharuskan UI melakukan *recomposition*. Akibatnya, tampilan `Text("Counter : $count")` tidak diperbarui secara otomatis meskipun nilai count telah berubah.

12. Apa yang dimaksud recomposition pada Compose?

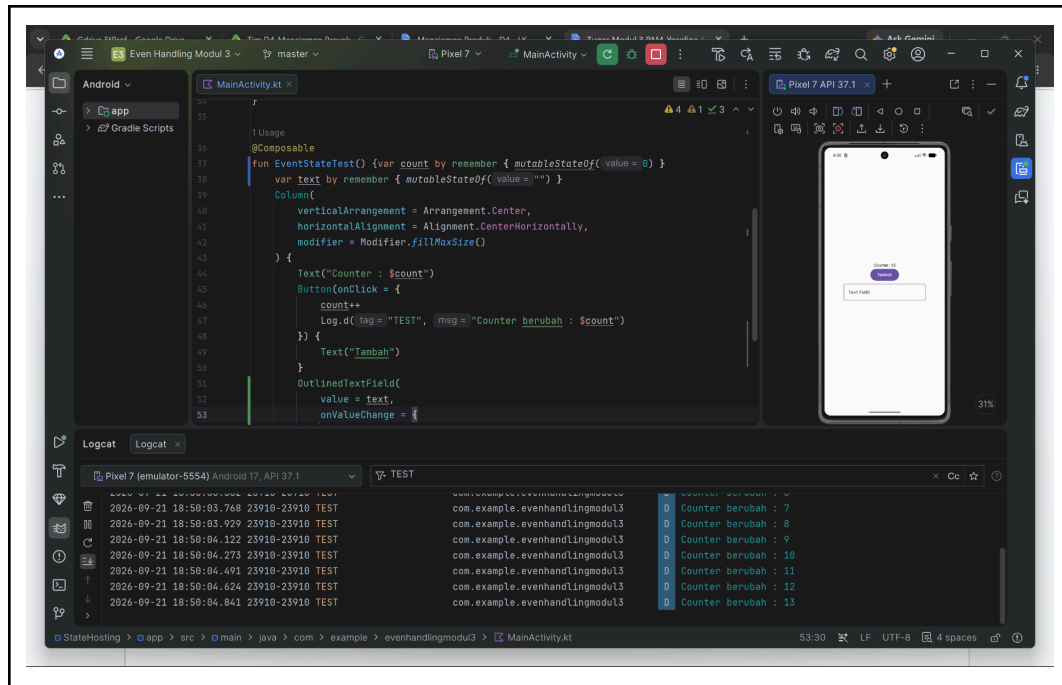
Recomposition pada Jetpack Compose adalah proses ketika Compose menjalankan ulang bagian UI yang datanya berubah agar tampilan sesuai dengan kondisi (*state*) terbaru. Recomposition terjadi ketika nilai *state* yang digunakan oleh suatu Composable berubah. Misalnya, jika nilai count berubah dari 0 menjadi 1 dan count disimpan menggunakan `mutableStateOf`, Compose

akan mendeteksi perubahan tersebut lalu menjalankan ulang bagian `Text("Counter : $count")`, sehingga teks pada layar berubah dari Counter: 0 menjadi Counter: 1 tanpa perlu memperbarui UI secara manual.

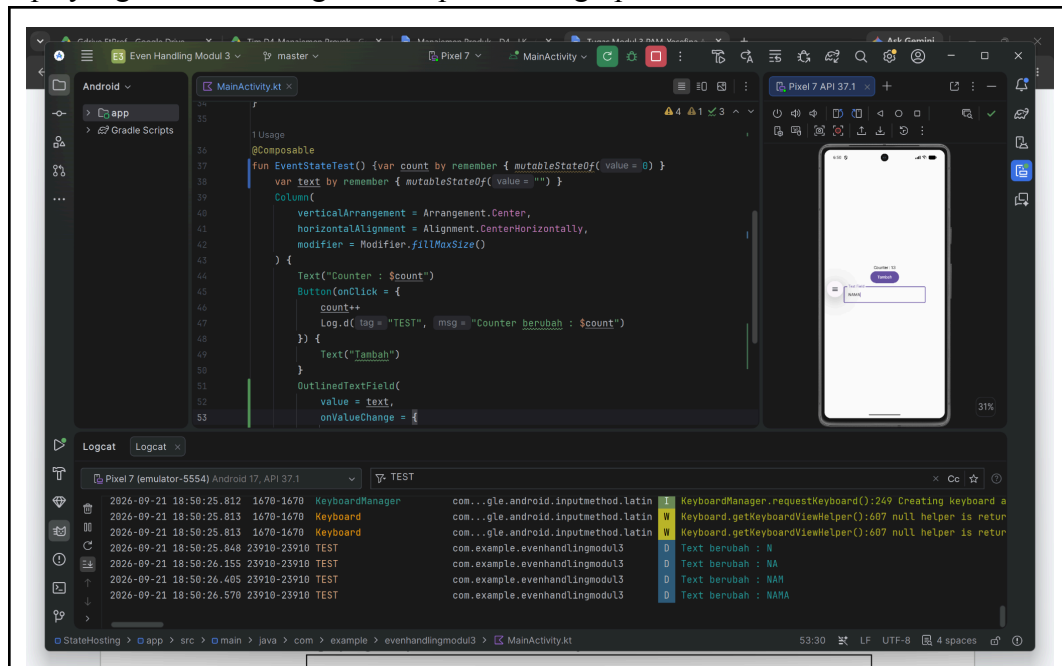
13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable `EventStateTest`. Kemudian di dalam Column panggil composable `OutlinedTextField` di bawah Button Tambah.



14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

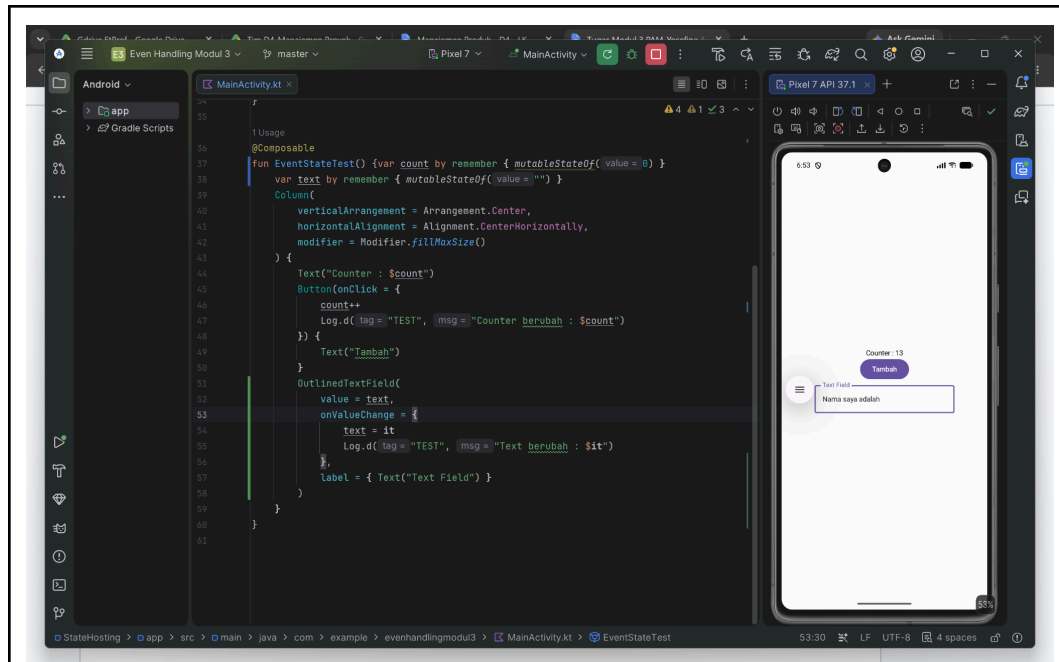


15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?



Logchat menulis huruf satu per satu.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?



Huruf yang diketik akan langsung muncul pada OutlinedTextField. Dan UI mengikuti nilai state text.

17. Apa yang menyebabkan hal tersebut terjadi?

Hal tersebut terjadi karena text dideklarasikan sebagai *state* menggunakan `var text by remember { mutableStateOf("") }`. Ketika pengguna mengetik pada TextField, `onValueChange` akan dipanggil dan nilai text diperbarui dengan nilai terbaru melalui `text = it`. Perubahan pada *state* text kemudian memicu proses recomposition, sehingga OutlinedTextField mendapatkan nilai `value = text` yang terbaru dan tampilan pada layar diperbarui secara otomatis sesuai dengan teks yang diketik oleh pengguna.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Ada 2 state:

1. `var count by remember { mutableStateOf(0) }`
2. `var text by remember { mutableStateOf("") }`

Dimana, count untuk menyimpan nilai counter, dan text untuk menyimpan nilai TextField.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Ada 2 event:

1. `onClick`
2. `onValueChange`

Dimana, `onClick` terdapat pada Button, sedangkan `onValueChange` terdapat

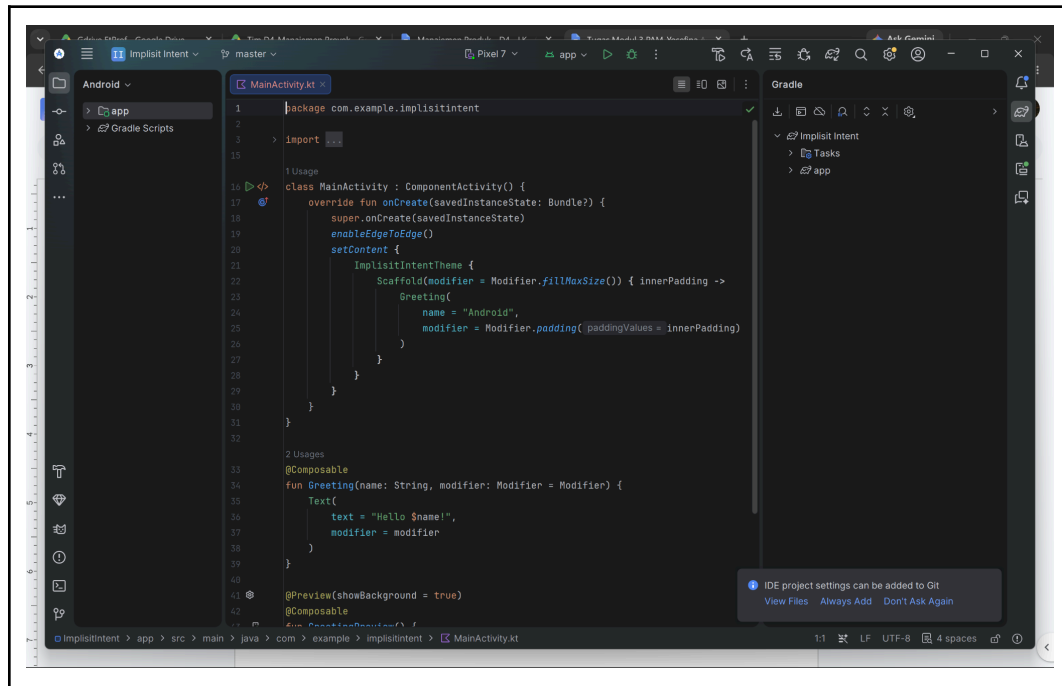
pada OutlinedTextField.

20. Kapan event - event tersebut dijalankan?

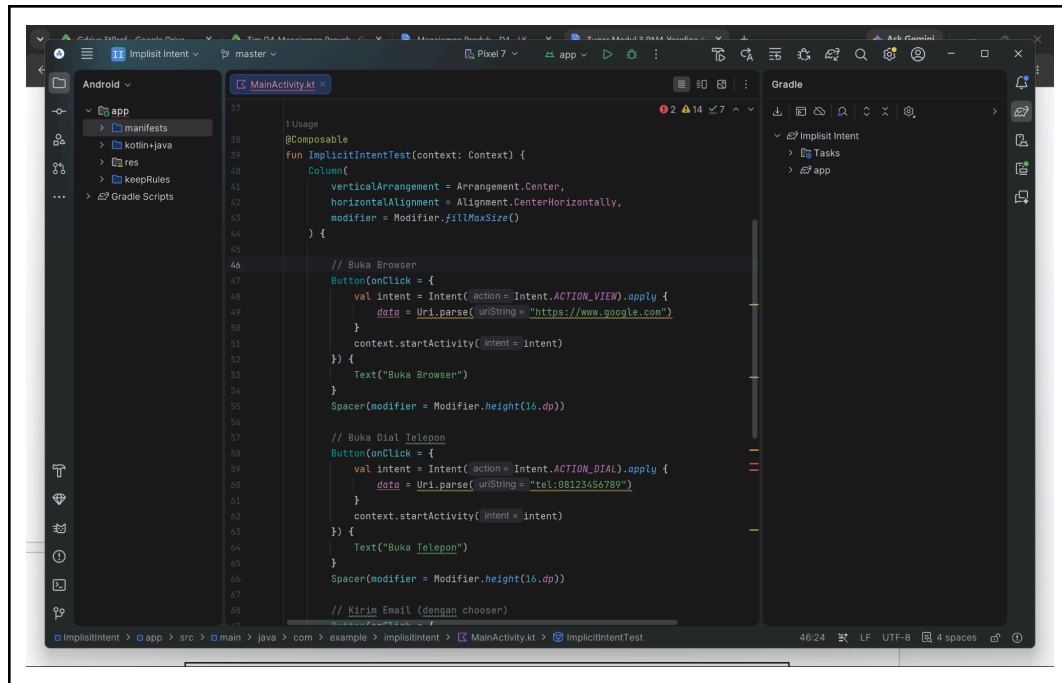
- a) onClick dijalankan ketika pengguna menekan tombol Tambah.
- b) onValueChange dijalankan setiap kali nilai/isi TextField berubah akibat input pengguna.

C. Implisit Intent

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.



2. Buka file MainActivity.kt dan tambahkan fungsi composable seperti yang diberikan.



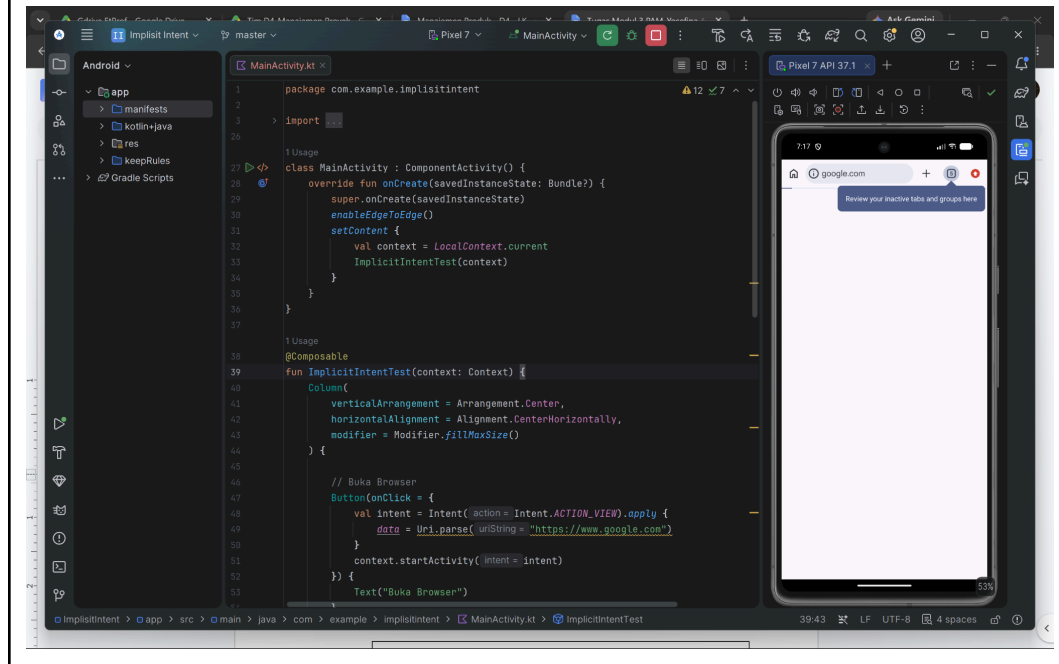
3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut.
4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } dan ubah sesuai kode yang ada.



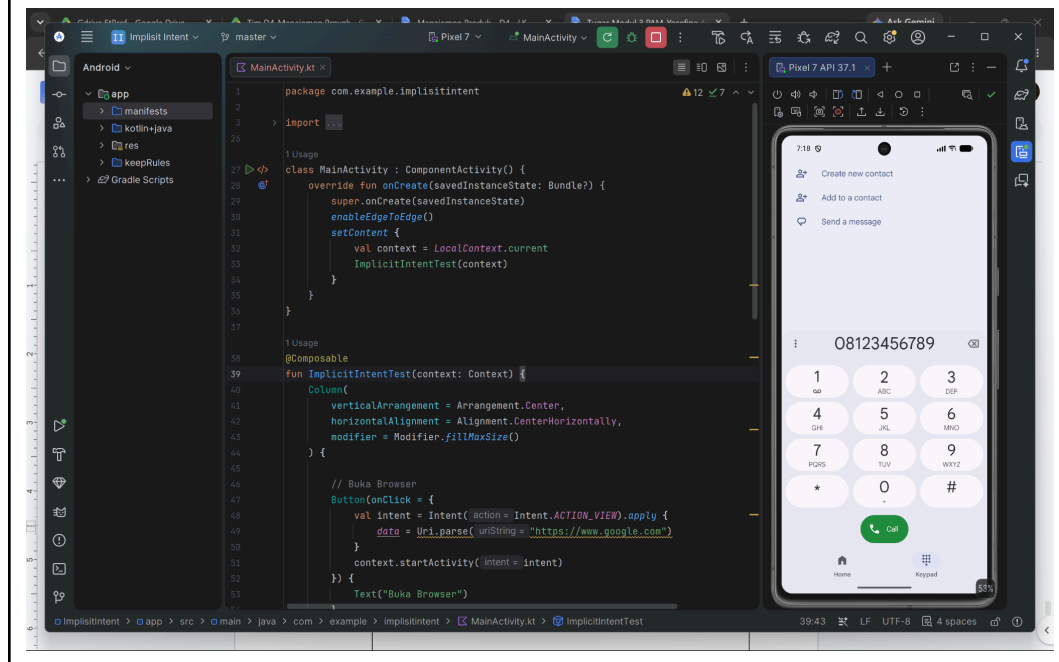
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol
 - Klik Buka Browser
 - Klik Buka Telepon
 - Klik Kirim Email
 - Klik Buka Maps
2. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

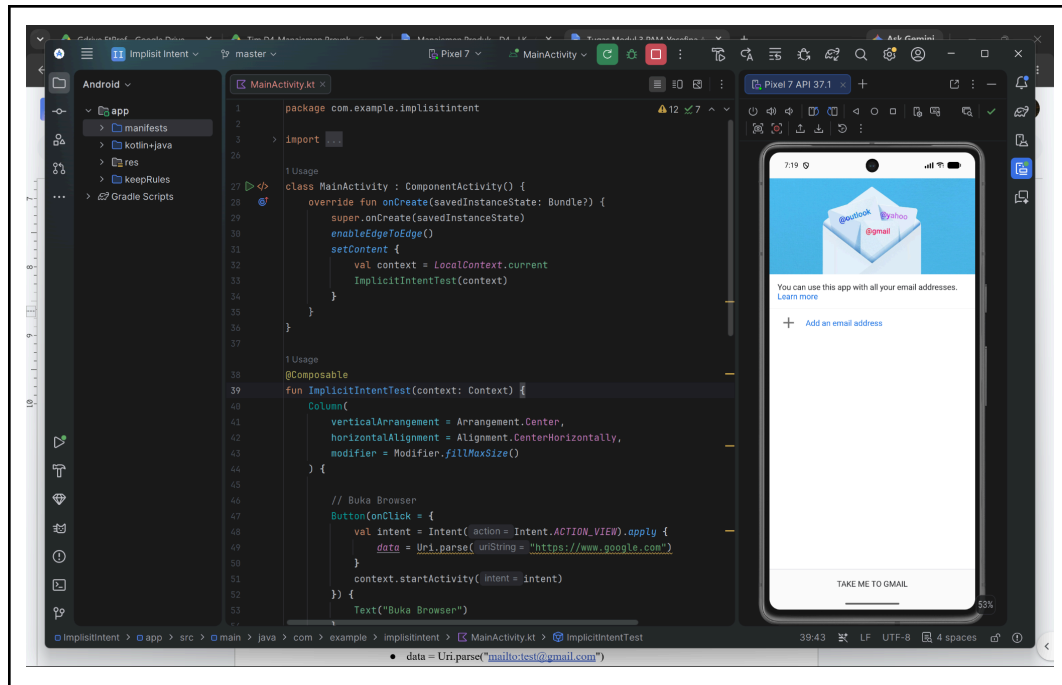
Buka Browser



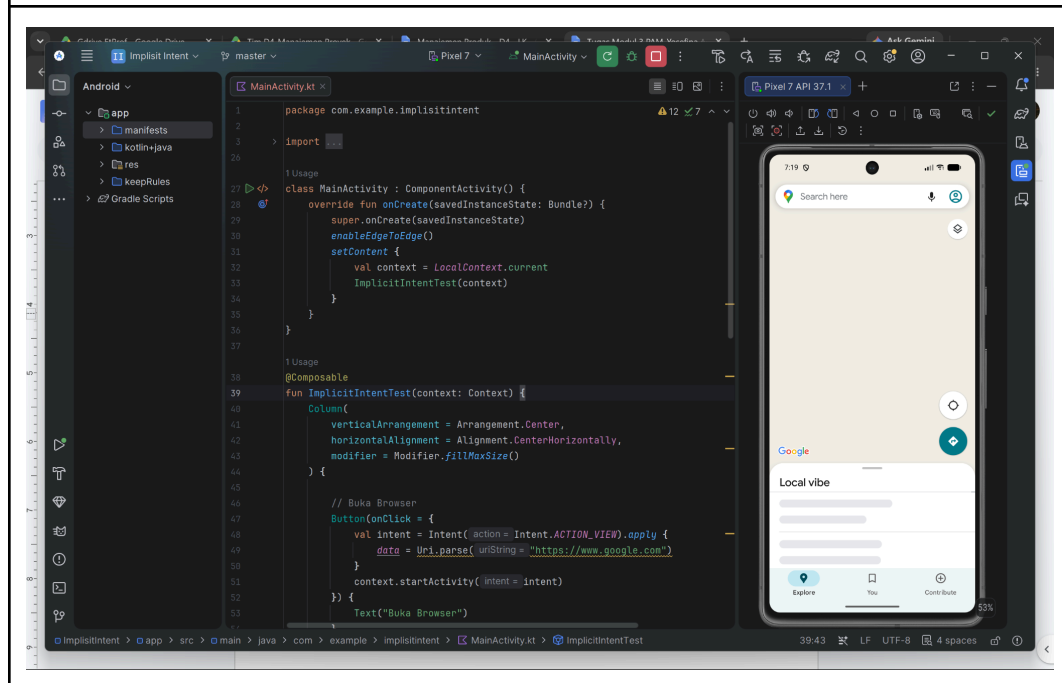
Buka Telepon



Kirim Email



Buka Maps



3. Pada kode yang digunakan, terdapat beberapa action seperti:
 - Intent.ACTION_VIEW
 - Intent.ACTION_DIAL
 - Intent.ACTION_SENDTO
4. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Perbedaan fungsi dari masing-masing *action intent* tersebut terletak pada jenis tindakan yang ingin dilakukan oleh aplikasi. Intent.ACTION_VIEW digunakan untuk menampilkan atau membuka suatu data, misalnya membuka halaman website melalui browser atau membuka lokasi melalui aplikasi Maps. Intent.ACTION_DIAL digunakan untuk membuka aplikasi telepon dan mengisi nomor tujuan secara otomatis, tetapi panggilan tidak langsung dilakukan sehingga pengguna tetap harus menekan tombol panggil.

Sementara itu, Intent.ACTION_SENDTO digunakan untuk mengirim data kepada tujuan tertentu, misalnya membuka aplikasi email dengan alamat penerima yang sudah ditentukan menggunakan URI mailto:. Dengan demikian, pemilihan *action* disesuaikan dengan tindakan yang ingin dilakukan, yaitu ACTION_VIEW untuk membuka atau melihat data, ACTION_DIAL untuk membuka dialer, dan ACTION_SENDTO untuk mengirim data ke tujuan tertentu.

5. Pada kode yang digunakan, terdapat beberapa data seperti:

- data = Uri.parse("<mailto:test@gmail.com>")
- data = Uri.parse("<https://www.google.com>")
- data = Uri.parse("geo:-7.250445,112.768845")

6. Apa fungsi data pada intent

data pada Intent berfungsi untuk menentukan data atau tujuan yang akan diproses oleh action tersebut. Data ini biasanya ditulis dalam bentuk Uri menggunakan Uri.parse(). Contohnya, mailto:test@gmail.com menunjukkan bahwa tujuan intent adalah alamat email test@gmail.com, https://www.google.com menunjukkan halaman web yang ingin dibuka, sedangkan geo:-7.250445,112.768845 menunjukkan koordinat lokasi yang ingin ditampilkan pada aplikasi peta. Jadi, action menentukan apa yang ingin dilakukan, sedangkan data menentukan terhadap data atau tujuan mana tindakan tersebut dilakukan.

7. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Action dan Data pada Intent saling berhubungan dalam menentukan aplikasi apa yang dapat digunakan untuk menjalankan Intent tersebut. Action menentukan tindakan yang ingin dilakukan, sedangkan Data menentukan jenis atau tujuan data yang akan diproses. Sistem Android akan membaca kombinasi keduanya, kemudian mencari aplikasi yang mampu menangani kombinasi tersebut. Misalnya, ACTION_VIEW dengan data berupa https://www.google.com akan memunculkan aplikasi yang dapat membuka halaman web, seperti browser. Sementara itu, ACTION_DIAL dengan data

tel:08123456789 akan diarahkan ke aplikasi telepon. Jika terdapat lebih dari satu aplikasi yang sesuai, Android dapat menampilkan pilihan aplikasi (*chooser*) kepada pengguna.

Link Github Keseluruhan Project:

[Tugas Pemesanan Tiket](#)

Output Tugas (Sebelum dan Sesudah di-klik)

